Vendor Management System (VMS)

Post-Hackathon Expansion & Implementation Roadmap

A dependency-ordered plan to evolve the Workday two-day hackathon MVP into a portfolio-grade contingent workforce, time approval, billing and payment platform.

Prepared	5 September 2026
Source basis	Uploaded workday-master code ZIP + embedded Vendor Management.pdf + consolidated code-review report + prior Workday VMS decisions
Current stack	React 19 / Vite / Tailwind + Node.js / Express + MySQL (mysql2, hand-written repositories)
Planning rule	No filler features; preserve strong transaction/idempotency patterns and add only workflow, trust, scalability or business-value capability.

1. Executive Direction

The current repository is already more than a CRUD hackathon app. It contains a working three-role staffing -> allocation -> daily timesheet -> approval -> milestone -> billing -> invoice flow with unusually careful transaction boundaries, row locks, idempotency and immutable financial snapshots. The expansion plan below deliberately preserves those strengths while correcting the biggest product gaps: incomplete approval context, weak contractor/client lifecycle, global vendor/project visibility, no compliance or date-based availability, invoice ownership semantics, missing payment tracking, and production hardening.

Target product: A vendor management platform that manages contingent-worker demand, candidate sourcing, compliance, assignment, approved work, billing eligibility, client invoice approval, payment tracking and offboarding.

Core pillars: Staffing -> Compliance -> Assignment -> Time & Approval -> Billing -> Invoice -> Payment -> Audit/Reporting.

Implementation philosophy: Improve the existing modular monolith; do not restart it. Every new feature must fit the current role, ownership, transaction and repository patterns.

2. Verified Current-State Baseline

Area	Verified baseline
Roles	VENDOR, CONTRACTOR, PM in one `users` table; Contractor self-signup disabled.
Client model	PM is linked to `client_companies` via `project_managers`; company is derived at project read time.
Projects	PM creates projects with date range, requirements and expected_hours; status ACTIVE/COMPLETED/ON_HOLD.
Contractors	Vendor creates/owns contractors; ACTIVE/INACTIVE, one primary skill, hourly_rate.
Assignment	Vendor directly fills requirement slots atomically; one active assignment per contractor via DB generated unique key; PM later sets allocated_hours.
Timesheets	Contractor logs one row per day; PENDING/APPROVED/REJECTED; rejected row can be edited/resubmitted; no rejection reason.
Milestones	Project-wide approved-hours thresholds; per-contractor milestone_billings snapshot approved hours/rate/amount.
Invoices	Automatically created from milestone_billings; current manual review belongs to Vendor; PM is read-only.
Dashboards	Vendor, PM and Contractor aggregated dashboards exist.
Engineering strengths	Transactions, row locking, deterministic lock order, ownership scoping, idempotency, immutable billing snapshots and honest migrations.
Engineering gaps	No formal test runner/CI, no pagination, no rate limiting/refresh/reset, no audit table, no structured logging/API docs.
Spec gaps	Rejection comments, client management, document verification, date-based availability, project budget, invoice number/tax/PDF and audit logging are missing.

3. Non-Negotiable Engineering Principles

- Preserve the current layered structure: validator -> controller -> service -> repository -> MySQL.
- Keep explicit transactions and SELECT ... FOR UPDATE on state-changing operations where race safety matters.

- Use database constraints/unique keys as the final concurrency guarantee; service pre-checks are for friendly errors only.
- Keep user/role identity derived from authenticated session/JWT; never trust vendor_id, pm_id, contractor_id ownership from request bodies.
- Continue the identical-404 pattern for cross-tenant resource probes where existence leakage matters.
- Financial and rate values that affect history must be snapshotted once and never recalculated from mutable master data.
- Prefer derived status/count metrics over stored duplicates when they can be computed reliably from authoritative records.
- Keep side effects such as notifications outside the transaction that commits the business mutation, unless the side effect itself is the durable record.
- Do not rewrite to microservices/Kubernetes/AI without a business requirement; the current modular monolith is appropriate.
- Every module exits only after manual E2E, automated tests, a short audit, documentation, commit and push.

4. Definition of Done for Every Module

- 1) Re-read the exact existing files/migrations that the module touches and write the intended invariant before coding.
- 2) Add versioned, idempotent MySQL migration(s) first; document any legacy-data limitation instead of fabricating history.
- 3) Implement repository -> service -> validator -> controller/route changes with ownership checks and transaction boundaries.
- 4) Implement the matching React service/page/component flow and explicit loading/empty/error states.
- 5) Manually test the role-to-role workflow end-to-end with both happy paths and abuse/edge cases.
- 6) Add/extend automated integration and frontend tests, including concurrency where state races matter.
- 7) Run a short audit: architecture, data integrity, authorization, security, concurrency/idempotency, backward compatibility and module integration.
- 8) Update README/API/CHANGELOG/ADR as needed, then commit and push that module before starting the next one.

5. Dependency-Ordered Roadmap at a Glance

ID	Phase	Priority	Module	Deps	Effort	Primary outcome
M00	PA - Preserve & harden	P0	Freeze Hackathon Baseline & Repository Governance	None	S	Preserve the two-day hackathon result as a verifiable baseline, then make every post-hackathon change traceable and deliberate.
M01	PA - Preserve & harden	P0	Automated Regression Suite & CI	M00	M	Turn the current ad-hoc live-server scripts into a repeatable safety net before changing business workflows.
M02	PA - Preserve & harden	P0	Authentication, Session & Account Security Hardening	M01	M	Replace hackathon-grade authentication with a production-

ID	Phase	Priority	Module	Deps	Effort	Primary outcome
						shaped session and account-recovery design.
M03	PA - Preserve & harden	P0	Structured Logging, Error Contracts & Operational Health	M01	S-M	Make failures diagnosable without exposing internals to users.
M04	PA - Preserve & harden	P0	Transactional Audit Logging Foundation	M01,M03	M	Create a real who-did-what-when trail for business-critical state changes.
M05	PA - Preserve & harden	P0	API Scalability, Search/Filtering & API Documentation	M01,M03	M	Make list endpoints scale beyond demo data and make the API contract explicit.
M06	PB - Workforce operations	P1	Timesheet Workflow Completion	M01,M04,M05	L	Complete the contractor -> submit -> PM review -> reject with reason -> correct -> resubmit workflow while preserving daily-row accuracy.
M07	PB - Workforce operations	P1	Contractor Profile & Multi-Skill Model	M01,M05	L	Move from one hard-coded skill enum to a credible contractor profile used by staffing decisions.
M08	PB - Workforce operations	P1	Contractor Document Verification & Compliance Gate	M04,M07	L	Implement document upload/review and prevent unverified contractors from being staffed.
M09	PB - Workforce operations	P1	Client Management, Vendor-Client Relationships & Project Visibility	M02,M04,M05	L	Turn client companies into an explicit business relationship and stop every vendor from seeing/staffing every project.
M10	PB - Workforce operations	P1	Project, Requirement, Budget & Time-Policy Lifecycle	M04,M09	L	Allow projects and staffing requirements to evolve safely after creation and add the missing commercial/time-policy fields.
M11	PB - Workforce operations	P1	Contractor Availability & Date-Based Assignment Lifecycle	M07,M08,M10	L	Replace the global one-ACTIVE-assignment rule with date-aware availability and controlled individual release.
M12	PB - Workforce	P1	Candidate	M09,M11	L	Replace direct

ID	Phase	Priority	Module	Deps	Effort	Primary outcome
	operations		Submission -> PM Acceptance Workflow			vendor assignment with a realistic propose-review-accept staffing flow.
M13	PB - Workforce operations	P1	Staffing Pipeline, Approval SLA & Escalation	M12,M10	M	Turn candidate records into operational staffing visibility and highlight items waiting too long.
M14	PB - Workforce operations	P1	In-App Notifications & Preferences	M06,M12,M13	M	Notify the right role when another role creates work that needs attention.
M15	PC - Commercial lifecycle	P1	Rate Cards, Assignment Rate Snapshots & Rate History	M09,M11	L	Separate commercial client bill rates from contractor cost/default rates and make rate changes historically safe.
M16	PC - Commercial lifecycle	P1	Milestone Management Enhancement	M06,M10,M15	M	Keep the proven project-wide hours trigger while making milestones understandable and operationally manageable.
M17	PC - Commercial lifecycle	P1	Invoice Lifecycle Redesign: Billing Queue -> Vendor Draft -> Client Approval	M09,M15,M16,M01	XL	Correct the commercial ownership model: milestones create billing eligibility; Vendor creates/submits an invoice; Client/PM approves or rejects it.
M18	PC - Commercial lifecycle	P1	Invoice Numbers, Line Items, Tax & PDF Document	M17	L	Make an approved/submitted invoice a real business document rather than a JSON row.
M19	PC - Commercial lifecycle	P1	Payment & Outstanding Tracking	M17,M18	M	Separate client approval from actual payment and expose outstanding/overdue money.
M20	PC - Commercial lifecycle	P2	Business Dashboards, Financial Health & CSV Exports	M05,M10,M13,M15,M19	L	Make dashboards reflect the full workforce/commercial lifecycle with precise terminology and exportable operational data.
M21	PD - Portfolio-grade operations	P2	Contractor Offboarding, History & Project Close Safeguards	M11,M19,M04	M	Close assignments/projects cleanly without losing historical

ID	Phase	Priority	Module	Deps	Effort	Primary outcome
						workforce or billing records.
M22	PD - Portfolio-grade operations	P2	Production Deployment, Backup, Security & Demo Environment	M01,M02,M03,M20	L	Make the expanded system reproducible and demonstrable outside the development machine.
M23	PD - Portfolio-grade operations	P2	Final Architecture Audit, Release & Internship Story	M00-M22	M	Validate that the expanded platform is coherent, tested and explainable, then cut a portfolio release.

6. Suggested Release Milestones

Release	Modules	Meaning
R0 / v1.0-hackathon	M00	Frozen historical baseline of the two-day submission.
R1 / v1.1-foundation	M01-M05	Tests, CI, security, logs, audit foundation, pagination and API docs.
R2 / v2.0-workforce	M06-M14	Complete time workflow, multi-skill/compliance, client/vendor scope, project/assignment lifecycle, candidate pipeline, SLA and notifications.
R3 / v3.0-commercial	M15-M20	Rate cards, milestone metadata, correct invoice lifecycle, real invoice documents, payment tracking, financial dashboards and exports.
R4 / v4.0-portfolio	M21-M23	Offboarding, deployment, recovery, end-to-end validation and internship-ready release.

7. Detailed Module-by-Module Implementation Plan

M00 - Freeze Hackathon Baseline & Repository Governance

Phase	Priority	Dependencies	Effort
Phase A - Preserve & harden	P0	None	S

Objective: Preserve the two-day hackathon result as a verifiable baseline, then make every post-hackathon change traceable and deliberate.

Business value: Protects the truth of what was built during the hackathon, makes later growth demonstrable in Git history, and prevents a rewrite from erasing the strongest existing engineering decisions.

Current code evidence / starting point

- README.md still documents only Module 1
- backend/src/migrations/001...016 show the actual incremental history
- backend/e2e_test.js is explicitly superseded while mvp_fix_test.js preserves a real billing regression

Implementation work

Layer	Implementation
Repository	Create Git tag/release `v1.0-hackathon` at the exact hackathon-complete commit; do not mix later refactors into that baseline.
Documentation	Rewrite root README as current-system documentation: roles, architecture, setup, implemented end-to-end workflow, demo accounts/data, and module map.
Documentation	Add `ARCHITECTURE.md`, `DATABASE.md`, `API.md`, `CHANGELOG.md`, `ROADMAP.md`, and a short ADR folder for major decisions.
Governance	Document invariants to preserve: JWT-derived identity, role-scoped routes, ownership checks in SQL, identical 404 for cross-tenant probes, explicit transactions/row locks, DB uniqueness as concurrency backstop, immutable billing snapshots, post-commit side effects.
Cleanup	Mark or remove obsolete verification-only routes/files only after equivalent real tests exist; retain historical notes where they explain superseded behavior.
Workflow	Adopt the permanent cycle: implement -> manual E2E -> short architecture/security audit -> automated tests -> commit/push -> next module.

Acceptance criteria

- Hackathon baseline is tagged and reproducible.
- A new developer can understand the current system from README/architecture docs without reading every source file.
- Every future module has an explicit changelog entry and commit boundary.

Scope guard: Do not migrate from raw mysql2 to an ORM just to match the original spec; the current repository/transaction model is a strength.

M01 - Automated Regression Suite & CI

Phase	Priority	Dependencies	Effort
Phase A - Preserve & harden	P0	M00	M

Objective: Turn the current ad-hoc live-server scripts into a repeatable safety net before changing business workflows.

Business value: The project has already had a real milestone-billing bug. Automated regression coverage is the highest-leverage protection against reintroducing financial and concurrency defects.

Current code evidence / starting point

- backend/mvp_fix_test.js
- backend/eligible_contractor_release_test.js
- backend/e2e_test.js
- backend/package.json has no test framework or CI script

Implementation work

Layer	Implementation
Backend	Add Jest + Supertest (or equivalent) and an isolated `vms_test` database configuration that refuses to run against a non-test DB.
Database	Run migrations automatically for tests; provide deterministic seed/factory helpers instead of relying on a manually prepared live database.
Regression	Port existing scenarios: RBAC/ownership, atomic assignment, no overstaffing, active-assignment conflict/release, PM allocation caps, daily timesheet uniqueness, rejected edit/resubmit, milestone trigger, per-contractor billing contribution, duplicate invoice protection.
Concurrency	Add tests that run simultaneous assignment/review/invoice operations and assert only one valid winner where uniqueness/conditional transitions apply.
Frontend	Add Vitest/React Testing Library for role routing, critical forms, rejection reason visibility, assignment state, timesheet state, and invoice state as modules evolve.
CI	Add GitHub Actions: install -> lint -> backend tests -> MySQL integration tests -> frontend tests -> production build.
Quality	Set a practical coverage floor for core services/repositories rather than chasing 100%; make failing tests block merges.

Acceptance criteria

- `npm test` runs without manually starting the API.
- CI creates an isolated MySQL service and passes from a clean checkout.
- The known milestone billing regression is permanently covered.
- A concurrency race cannot silently pass as two successful mutations.

Scope guard: Do not delete the historical scripts until their business scenarios are represented in the automated suite.

M02 - Authentication, Session & Account Security Hardening

Phase	Priority	Dependencies	Effort
Phase A - Preserve & harden	P0	M01	M

Objective: Replace hackathon-grade authentication with a production-shaped session and account-recovery design.

Business value: Protects vendor, client and contractor accounts that control staffing, timesheets and money; removes brute-force and long-lived bearer-token weaknesses.

Current code evidence / starting point

- backend/src/routes/authRoutes.js exposes signup/login/me only
- backend/src/services/authService.js issues one JWT
- frontend/src/utils/tokenStorage.js stores the token client-side
- backend/src/app.js has CORS + express.json but no rate limiter/security headers

Implementation work

Layer	Implementation
Security	Add `helmet`, strict CORS allowlist by environment, request body limits, auth-specific rate limiting and progressive account lockout/backoff.
Sessions	Move to short-lived access tokens plus refresh/session records with rotation, revocation and logout-all; prefer an HttpOnly Secure SameSite refresh cookie rather than a long-lived localStorage bearer token.
Recovery	Implement forgot-password/reset-password using single-use hashed reset tokens with expiry; enforce the same password policy as signup.
Contractor onboarding	Replace vendor-chosen permanent passwords with invitation/first-login password setup. Keep a temporary compatibility path only while migrating seed/demo users.
Validation	Reject unknown auth fields; normalize email consistently; keep generic login/recovery responses that do not reveal account existence.
Frontend	Add session restoration, expired-session handling, password setup/reset screens, logout-all entry point, and clear unauthorized vs expired states.
Tests	Cover brute-force limits, token rotation/reuse rejection, logout revocation, reset-token expiry/reuse, and contractor invitation acceptance.

Acceptance criteria

- Repeated failed logins are rate-limited/locked safely.
- A stolen/expired access token can be revoked via the session model.
- Forgot-password works without leaking whether an email exists.
- Vendor-created contractors set their own password on first login.

Scope guard: Do not add social login/SSO yet; it adds integration breadth without improving the core VMS workflow.

M03 - Structured Logging, Error Contracts & Operational Health

Phase	Priority	Dependencies	Effort
Phase A - Preserve & harden	P0	M01	S-M

Objective: Make failures diagnosable without exposing internals to users.

Business value: When a staffing, approval or billing operation fails, developers need to trace one request across controller/service/repository boundaries and know whether the transaction committed.

Current code evidence / starting point

- backend/src/server.js and services use console.log/console.error
- backend/src/middleware/errorHandler.js is the central error boundary
- /api/health currently returns only a simple ok response

Implementation work

Layer	Implementation
Logging	Introduce structured JSON logs with request_id/correlation_id, route, actor user id/role, status, duration and stable error code; redact passwords/tokens/document data.
Middleware	Generate/accept a request ID at ingress and return it in response headers/errors.
Errors	Standardize API errors as `{code,message,details?,request_id}`; map DB duplicate/constraint failures to stable domain codes.
Health	Split liveness/readiness semantics: process live, DB ready, optional dependency status.
Metrics	At minimum record counts/latency for auth failures,

Layer	Implementation
	assignment conflicts, timesheet reviews, billing triggers and invoice transitions.
Tests	Assert no stack traces/secrets reach production responses and request IDs appear consistently.

Acceptance criteria

- A production error can be traced by request ID.
- Client responses use stable domain error codes.
- Logs contain no raw passwords/JWTs/document contents.

Scope guard: Do not introduce a full distributed tracing stack before the app actually has multiple services.

M04 - Transactional Audit Logging Foundation

Phase	Priority	Dependencies	Effort
Phase A - Preserve & harden	P0	M01,M03	M

Objective: Create a real who-did-what-when trail for business-critical state changes.

Business value: Hours, contractor assignment, rates and invoices are dispute-sensitive. An immutable audit trail is core product value, not decorative logging.

Current code evidence / starting point

- No audit table exists in migrations 001-016
- reviewed_by/reviewed_at exist only on selected timesheet/invoice rows
- recent dashboard activity is reconstructed from operational tables rather than a durable event trail

Implementation work

Layer	Implementation
Database	Add `audit_log(id, actor_user_id, actor_role, action, entity_type, entity_id, before_json, after_json, request_id, created_at)` with indexes on entity and actor/time.
Backend	Add `auditRepository`/`auditService`; writes must accept the caller transaction connection so the business mutation and audit record commit/rollback together.
Coverage	Instrument assignment create/release/allocation, project/requirement changes, timesheet submit/edit/review, document verification, candidate decisions, rate changes, milestone changes, invoice transitions and payment records as those modules ship.
Security	Redact secrets and large binary/document content; audit references metadata/IDs only.
Frontend	Add an entity-level timeline drawer later for authorized Vendor/PM users; do not expose cross-tenant audit entries.
Tests	Prove rollback removes the audit row, successful mutation creates exactly one audit record, and before/after payloads match the committed state.

Acceptance criteria

- Every critical mutation has an audit record with actor and timestamp.
- Audit writes are transactionally consistent with the business change.
- Cross-tenant users cannot query another tenant/project audit history.

Scope guard: Audit log is append-only through application APIs; do not add general edit/delete endpoints.

M05 - API Scalability, Search/Filtering & API Documentation

Phase	Priority	Dependencies	Effort
Phase A - Preserve & harden	P0	M01,M03	M

Objective: Make list endpoints scale beyond demo data and make the API contract explicit.

Business value: Vendors and PMs will accumulate contractors, projects, timesheets and invoices quickly; unbounded lists become slow and unusable.

Current code evidence / starting point

- vendor/pm/contractor list services return complete result sets
- invoiceRepository.listForVendor/listForPm are unpaginated
- No OpenAPI/Swagger or checked-in Postman collection exists

Implementation work

Layer	Implementation
API	Adopt consistent query parameters: `page`, `pageSize`, `sort`, `order`, plus domain filters. Return `{items,page,page_size,total,total_pages}`.
Database	Add/verify composite indexes supporting vendor+status/skill, project+status/date, timesheet+project/status/date, invoice+vendor/status/date and candidate pipeline later.
Search	Contractors: name/email/skill/status; projects: name/client/status/date; timesheets: project/contractor/status/date; invoices: project/status/date/number when introduced.
Backend	Move filtering/pagination into repository SQL; never fetch-all then filter in JavaScript.
Frontend	Reusable table/list filter bar, debounced text search, URL-synced filters, empty/loading/error states and pagination controls.
Docs	Add OpenAPI specification and generated/checked API examples for every route; document auth requirements and error codes.
Tests	Verify pagination totals, filter combinations, stable sort, authorization scoping and index-friendly query shapes.

Acceptance criteria

- No main list endpoint is unbounded.
- Filters remain tenant-scoped in SQL.
- API documentation matches implemented routes and payloads.

Scope guard: Do not create a generic query-builder abstraction that obscures SQL ownership rules; keep repository queries explicit.

M06 - Timesheet Workflow Completion

Phase	Priority	Dependencies	Effort
Phase B - Workforce operations	P1	M01,M04,M05	L

Objective: Complete the contractor -> submit -> PM review -> reject with reason -> correct -> resubmit workflow while preserving daily-row accuracy.

Business value: This closes the clearest original-spec gap and makes time approval operational rather than a set of isolated daily status flips.

Current code evidence / starting point

- timesheets are daily rows after migration 013

- current status is PENDING/APPROVED/REJECTED
- pmTimesheetValidators accepts only bare status; no rejection_reason column
- frontend already groups daily rows into weeks via weekGrouping.js

Implementation work

Layer	Implementation
Database	Add `description`, `rejection_reason`, explicit `submitted_at`; migrate status enum to `DRAFT,SUBMITTED,APPROVED,REJECTED` and map existing PENDING -> SUBMITTED.
Contractor API	Allow save/edit DRAFT entries; add submit-week/submit-selected action that atomically transitions owned DRAFT/REJECTED rows to SUBMITTED after validation.
PM API	List SUBMITTED rows; require non-blank rejection reason for REJECTED; add bulk approve/reject for selected entries with deterministic lock ordering by timesheet id.
Business rules	Keep project/date/allocation validation server-side; APPROVED rows immutable; rejected rows editable by owner; resubmission clears prior review fields while preserving audit history.
Policies	Retain current 24h/day ceiling initially; add hooks for project-specific daily/weekly/weekend/backdate policies in M10 rather than hard-coding a universal 40h week.
Frontend	Weekly timesheet workspace: day rows, descriptions, draft state, submit-week action, rejection reason banner, edit/resubmit path, PM bulk review table.
Billing integration	Milestone evaluation runs only after APPROVED transitions; draft/submitted/rejected hours never become billable.
Tests	State-transition matrix, rejection reason required, weekly submit ownership, duplicate date, allocation capacity with DRAFT/SUBMITTED reservations, bulk review race safety, resubmission audit.

Acceptance criteria

- Contractor can save a week as draft and explicitly submit it.
- PM rejection always explains why.
- Contractor can correct and resubmit only rejected work.
- Only approved hours affect milestone/billing.

Scope guard: Do not replace the daily timesheet table with an opaque weekly total; daily entries are needed for auditability and policy validation.

M07 - Contractor Profile & Multi-Skill Model

Phase	Priority	Dependencies	Effort
Phase B - Workforce operations	P1	M01,M05	L

Objective: Move from one hard-coded skill enum to a credible contractor profile used by staffing decisions.

Business value: Real contractors can have multiple relevant skills and experience; single-skill matching creates false shortages and weak candidate context.

Current code evidence / starting point

- migration 005 adds a single nullable contractors.skill enum
- contractor profile route only reads/updates the primary skill
- Vendor contractor view exposes name/email/rate/status/skill only

Implementation work

Layer	Implementation
Database	Add `skills` catalog and `contractor_skills(contractor_id, skill_id, proficiency, years_experience, is_primary)`; backfill current FRONTEND/BACKEND/QA/DEVOPS/DATA values.
Migration	Add `skill_id` to project requirements, backfill from existing enum, update queries, then drop/retire enum columns only after parity tests.
Profile	Add phone, headline/role, total experience years and optional notes; keep only business-relevant fields.
Backend	Repositories/services for skill catalog and contractor skill CRUD; vendor can manage profile fields, contractor can maintain allowed self-service fields based on a documented ownership matrix.
Matching	Eligible-contractor queries match any verified contractor skill rather than one equality field; primary skill is presentation only.
Frontend	Contractor detail/profile with skill chips, proficiency, experience; vendor filters by multiple skills.
Tests	Backfill parity, duplicate skill prevention, ownership, skill add/remove effect on future eligibility only.

Acceptance criteria

- A contractor can hold multiple skills with proficiency/experience.
- Existing requirements continue matching after migration.
- Changing skills never rewrites historical assignment/billing records.

Scope guard: Do not add AI skill inference or resume scoring; skills are explicit user-managed business data.

M08 - Contractor Document Verification & Compliance Gate

Phase	Priority	Dependencies	Effort
Phase B - Workforce operations	P1	M04,M07	L

Objective: Implement document upload/review and prevent unverified contractors from being staffed.

Business value: Directly replaces manual document verification from the original problem statement and prevents assigning workers who have not passed required compliance checks.

Current code evidence / starting point

- Original spec requires contractor documents and PENDING/VERIFIED/REJECTED status
- No contractor_documents table or upload code exists
- vendorAssignmentService currently checks ACTIVE + skill + active-assignment conflict only

Implementation work

Layer	Implementation
Database	Add `contractor_documents` with type, storage key/url, status PENDING/VERIFIED/REJECTED/EXPIRED, expiry_date, verified_by, verified_at, rejection_reason, timestamps.
Policy	Define a small required-document catalog (e.g. ID, PAN/tax, qualification/experience as configured) and derive overall contractor compliance status from required docs.
Storage	Create a storage abstraction; local/dev driver for portfolio use and object-storage compatible interface for production. Persist metadata, not binaries, in MySQL.
Security	Validate extension/MIME/content signature, size limits and authorization; use signed/authorized downloads, never public raw paths.
Backend	Vendor uploads/manages docs; authorized PM/client may view verification summary but not unrelated vendor private

Layer	Implementation
	documents. Verification action writes audit record.
Eligibility	Assignment/candidate eligibility rejects contractors whose required compliance status is not VERIFIED or whose required document expires before/during assignment dates.
Frontend	Compliance checklist, upload status, rejection reason, expiry warnings, verification queue.
Tests	Unauthorized file access, invalid files, status transitions, expiry gating and assignment rejection until verified.

Acceptance criteria

- Unverified contractor cannot be assigned/accepted.
- Verification decision is auditable and rejection reason is visible.
- Expired mandatory documents remove future eligibility without deleting history.

Scope guard: Do not implement OCR/AI document verification initially; human verification with explicit status is the correct first version.

M09 - Client Management, Vendor-Client Relationships & Project Visibility

Phase	Priority	Dependencies	Effort
Phase B - Workforce operations	P1	M02,M04,M05	L

Objective: Turn client companies into an explicit business relationship and stop every vendor from seeing/staffing every project.

Business value: This is both a real client-management feature and a multi-tenant boundary. In a real VMS, a vendor should only see projects they are authorized to supply.

Current code evidence / starting point

- migrations 009/010 create client_companies + project_managers
- authService auto-links PM signup to a company name
- vendor project workflow currently has no vendor-project/client relationship and intentionally allowed any Vendor to staff any project in the MVP

Implementation work

Layer	Implementation
Database	Add `client_vendor_relationships(client_company_id,vendor_id,status,invited_by,accepted_at,...)` and `project_vendors(project_id,vendor_id,status,...)` for project-level sourcing access.
Onboarding	Stop unrestricted company-name claiming: new PM membership should require an invitation/verified company membership flow once a company exists; preserve a controlled first-company bootstrap path for demo/admin setup.
PM flow	PM can invite/select connected vendor(s) for a project; revoke future sourcing without deleting historical assignments.
Vendor flow	Vendor Clients page shows connected companies, PM contacts, active projects, open requirements, deployed contractors and invoice/payment summary once available.
Authorization	Vendor `/projects`, requirement and candidate queries must join through active `project_vendors`; unauthorized project IDs return the same 404 pattern used elsewhere.
Frontend	Client directory/detail pages for Vendor; vendor access management within PM project view.
Tests	Two vendors/two clients matrix proving no cross-project visibility or staffing outside an active relationship.

Acceptance criteria

- Vendor sees only authorized client/project demand.
- PM can connect/remove a vendor from future sourcing.
- Vendor has a real client-management view without turning the project into a generic CRM.

Scope guard: Do not add a broad CRM pipeline, sales leads, marketing contacts or unrelated account-management features.

M10 - Project, Requirement, Budget & Time-Policy Lifecycle

Phase	Priority	Dependencies	Effort
Phase B - Workforce operations	P1	M04,M09	L

Objective: Allow projects and staffing requirements to evolve safely after creation and add the missing commercial/time-policy fields.

Business value: Real projects change dates, budget and staffing demand. Safe edits are more valuable than forcing users to recreate projects and lose history.

Current code evidence / starting point

- projects currently have dates/status/expected_hours but no budget/currency
- no general project edit endpoint exists
- requirements are created at project creation and have no lifecycle/update endpoint
- projects.status already has ACTIVE/COMPLETED/ON_HOLD

Implementation work

Layer	Implementation
Database	Add `budget DECIMAL`, `currency CHAR(3)`, optional `max_hours_per_day`, `max_hours_per_week`, `allow_weekend`, `backdate_limit_days`; add requirement `status OPEN/CLOSED` and optional description.
Project API	Add controlled PATCH for name/description/dates/budget/expected_hours/time policies/status; validate against committed data.
Safety	Cannot reduce expected_hours below allocated or approved hours; cannot move dates so existing assignments/timesheets fall outside; cannot complete while there are submitted timesheets or unresolved billing eligibility without an explicit resolution path.
Requirement API	Increase/decrease required_count, close/reopen requirement. Required count cannot drop below already accepted/assigned contractors; do not delete historical requirements.
Status	Define explicit transitions ACTIVE <-> ON_HOLD, ACTIVE/ON_HOLD -> COMPLETED, ACTIVE/ON_HOLD -> CANCELLED (new), with side effects documented.
Timesheet integration	M06 validation reads project-specific policy values; ON_HOLD/CANCELLED/COMPLETED block new submissions.
Frontend	Project settings/edit drawer, requirement editor, budget + policy cards, clear blocked-edit reasons.
Tests	Every invalid date/hours/budget transition, requirement decrease, status transition and existing-data safeguard.

Acceptance criteria

- Project can be edited without corrupting assignments/timesheets.
- Budget and currency exist and are available to reporting.
- Requirements can be managed after creation with safe lower bounds.

Scope guard: Do not allow destructive project deletion once business records exist; use lifecycle states and retain history.

M11 - Contractor Availability & Date-Based Assignment Lifecycle

Phase	Priority	Dependencies	Effort
Phase B - Workforce operations	P1	M07,M08,M10	L

Objective: Replace the global one-ACTIVE-assignment rule with date-aware availability and controlled individual release.

Business value: A contractor should be able to finish one project and be pre-booked for another without overlap; vendors need to know actual future availability.

Current code evidence / starting point

- migration 016 uses active_contractor_key UNIQUE to permit one ACTIVE assignment at a time
- project_assignments has assigned_date but no assignment start/end range
- project completion releases all active assignments; no individual release route exists

Implementation work

Layer	Implementation
Database	Add assignment `start_date,end_date,actual_end_date,release_reason,rate_card_id/rate snapshot later`; add `contractor_unavailability(contractor_id,start_date,end_date,reason,status)`.
Migration	Remove the generated one-active-assignment uniqueness only after a transactionally safe overlap query/locking strategy and tests exist.
Conflict rule	Before candidate acceptance/assignment, reject any date overlap with another non-released assignment; lock contractor + relevant assignment range inside the transaction.
Availability	Vendor/contractor can record planned unavailable ranges; eligibility rejects overlapping unavailability.
Release	Add authorized individual release with actual last working date and reason; historical assignment stays intact. Release frees future eligibility.
PM allocation	Allocated hours remain project-specific; validate allocation against assignment dates and project capacity.
Frontend	Availability calendar/list, assignment dates, release action, conflict explanation and upcoming assignment display.
Tests	Boundary dates (touching vs overlapping), concurrent acceptance, release/reassign, unavailability conflicts, project completion releasing remaining active assignments.

Acceptance criteria

- Contractor can hold sequential future assignments without overlap.
- Overlapping assignments are rejected race-safely.
- One contractor can be released without completing the project.

Scope guard: Do not support fractional overlapping allocations until there is a validated business need; first make no-overlap date scheduling correct.

M12 - Candidate Submission -> PM Acceptance Workflow

Phase	Priority	Dependencies	Effort
Phase B - Workforce operations	P1	M09,M11	L

Objective: Replace direct vendor assignment with a realistic propose-review-accept staffing flow.

Business value: Separates vendor sourcing from client decision-making and makes requirement fulfillment defensible, reviewable and auditable.

Current code evidence / starting point

- Current vendor route POST .../assign directly creates assignments

- Original workflow has vendor managing contractors and client validating work; a real VMS normally distinguishes candidate proposal from assignment

Implementation work

Layer	Implementation
Database	Add `candidate_submissions` with project, requirement, contractor, vendor, proposed dates/rate, status SUBMITTED/SHORTLISTED/ACCEPTED/REJECTED/WITHDRAWN, vendor_note, pm_note, timestamps/reviewer.
Vendor API	Submit eligible contractor to an authorized requirement; withdraw only before acceptance; prevent duplicate active submission for same contractor+requirement.
PM API	Review candidate, shortlist, accept or reject with reason. Acceptance runs in one transaction and rechecks project/vendor access, open requirement capacity, compliance, skill, date availability and conflicts.
Assignment	Only ACCEPTED candidate creates the project_assignment row; retire direct vendor assignment from normal UI after migration.
Rate	Snapshot proposed/selected assignment rate or rate-card reference at acceptance once M15 is available; until then preserve current hourly_rate behavior with explicit migration note.
Frontend	Vendor requirement candidate picker + submission status; PM candidate queue/detail + accept/reject; contractor sees assignment only after acceptance.
Audit	Every submission/status transition is logged.
Tests	Double-submit, two PM acceptance races on last slot, compliance expiry between submit/accept, vendor-project access revocation, withdrawal and rejection.

Acceptance criteria

- Vendor cannot directly place a worker into a client project.
- PM acceptance is the only normal path that creates an assignment.
- Last-slot and overlap races cannot over-assign.

Scope guard: Do not add algorithmic candidate ranking; filters and explicit business criteria are sufficient.

M13 - Staffing Pipeline, Approval SLA & Escalation

Phase	Priority	Dependencies	Effort
Phase B - Workforce operations	P1	M12,M10	M

Objective: Turn candidate records into operational staffing visibility and highlight items waiting too long.

Business value: The real pain is not just “pending”; it is knowing where staffing is blocked and what has breached an agreed response time.

Current code evidence / starting point

- Current requirement view mainly shows required_count vs assigned_count
- Dashboards already compute derived staffing progress but no candidate funnel/SLA exists

Implementation work

Layer	Implementation
Derived metrics	For each requirement derive available/submitted/shortlisted/accepted/assigned/open counts; do not store counts that can drift.
SLA policy	Add project/client setting for candidate_response_sla_hours and later reuse a generic approval SLA model for

Layer	Implementation
	timesheets/invoices.
Status	Compute `due_at`/breach at read time from submitted_at + policy; optionally persist escalation acknowledgement, not the derived breach itself.
Dashboard	Vendor sees requirements with stalled submissions; PM sees oldest candidate reviews and open demand.
Filters	By client/project/skill/status/SLA breached.
Tests	Correct funnel counts under rejection/withdrawal/acceptance, SLA timezone boundaries and access scoping.

Acceptance criteria

- One screen answers how many positions are still open and where candidates are stuck.
- SLA breach is based on explicit policy/time, not a fabricated status.

Scope guard: Do not create performance scores from SLA counts; report the operational facts only.

M14 - In-App Notifications & Preferences

Phase	Priority	Dependencies	Effort
Phase B - Workforce operations	P1	M06,M12,M13	M

Objective: Notify the right role when another role creates work that needs attention.

Business value: This platform is a chain of handoffs. Without notifications, approvals still depend on manual checking and the system recreates the email/spreadsheet problem it was meant to replace.

Current code evidence / starting point

- No notification model/service exists
- Existing service design already uses post-commit hooks (e.g. milestone evaluation after timesheet approval) suitable for non-transactional side effects

Implementation work

Layer	Implementation
Database	Add `notifications` and `notification_preferences` (event type, in-app enabled, email later, quiet hours optional later).
Backend	Create notification service called only after business commit; notification failure must never roll back staffing/timesheet/invoice state.
Events	Start with candidate submitted/accepted/rejected, assignment created/released, timesheet submitted/approved/rejected, document expiring, milestone met/billing eligible. Extend invoice/payment events in M17-M19.
Frontend	Bell/inbox, unread count, mark read/all read, deep links to relevant entity, per-event preference page.
Privacy	Keep messages generic enough for shared screens; do not include sensitive document content.
Reliability	For portfolio scale, persisted in-app notification rows are sufficient; add an outbox/worker only if deployment reliability later requires it.
Tests	No duplicate notification on retry/idempotent mutation, correct recipient role, deep-link authorization.

Acceptance criteria

- A user can see actionable state changes without manually polling every page.
- Notification failure cannot undo a committed business transaction.
- Users can disable non-critical event types.

Scope guard: Do not add push/email/SMS channels all at once; build one reliable in-app channel first.

M15 - Rate Cards, Assignment Rate Snapshots & Rate History

Phase	Priority	Dependencies	Effort
Phase C - Commercial lifecycle	P1	M09,M11	L

Objective: Separate commercial client bill rates from contractor cost/default rates and make rate changes historically safe.

Business value: A real vendor needs client-specific rates and margin visibility; a contractor master rate alone cannot represent negotiated project pricing.

Current code evidence / starting point

- contractors.hourly_rate is the only current rate source
- milestone_billings snapshots hourly_rate and billing_amount immutably
- Historical snapshot behavior is already a strong pattern to preserve

Implementation work

Layer	Implementation
Database	Add `rate_cards(client_company_id,vendor_id,skill_id,effective_from,effective_to,bill_rate,cost_rate,currency,status)`; add assignment `bill_rate_snapshot,cost_rate_snapshot,currency,rate_card_id`.
Compatibility	Keep contractors.hourly_rate as a legacy/default rate initially; do not rewrite historical milestone_billings.
Resolution	At candidate acceptance/assignment, resolve an active rate card (or explicit approved override) and snapshot rates onto the assignment.
Rules	Rate changes affect future/new assignments or explicit future effective periods only; approved historical work always uses the assignment/billing snapshot.
Backend	Rate-card CRUD scoped to vendor+client relationship, effective-date overlap validation, audit every rate change.
Frontend	Vendor rate-card page by client/skill, effective dates, current/upcoming rates; PM can view agreed bill rate where authorized.
Tests	Effective-date boundary, overlapping rate-card rejection, historical snapshot immutability, currency consistency.

Acceptance criteria

- Same skill can have different rates for different clients/projects.
- Historical billing never changes when a rate card changes.
- Both bill rate and cost rate are available for later margin analytics.

Scope guard: Do not attempt payroll calculation; contractor cost rate is only a commercial input for project/vendor margin in this VMS.

M16 - Milestone Management Enhancement

Phase	Priority	Dependencies	Effort
Phase C - Commercial lifecycle	P1	M06,M10,M15	M

Objective: Keep the proven project-wide hours trigger while making milestones understandable and operationally manageable.

Business value: Current hour thresholds work technically, but PMs need milestone context, due dates and visibility into whether billing eligibility was created.

Current code evidence / starting point

- milestones are project-level after migration 016
- current fields are mainly name, threshold_hours, status PENDING/MET
- milestone_billings already provide per-contractor immutable contribution snapshots

Implementation work

Layer	Implementation
Database	Add milestone description, sequence/order, due_date and optional status metadata needed for display; retain threshold_hours as the trigger source.
Lifecycle	Keep threshold evaluation deterministic from approved project hours. Expose PENDING/MET and derive billing/invoicing completeness from contribution records rather than duplicating financial truth.
Editing	Allow metadata edits while PENDING; threshold can only change if it does not invalidate already met/approved history. MET milestones become immutable except descriptive notes.
Frontend	Project milestone timeline with approved-hours progress, due date, contribution breakdown and billing eligibility state.
Billing	Continue creating per-contractor milestone_billings exactly once; use assignment rate snapshots from M15 for new billings.
Tests	Threshold edits, already-met immutability, multiple contractors, repeated evaluation idempotency, late-created milestone already below approved hours.

Acceptance criteria

- Milestones are understandable business checkpoints without weakening the current idempotent billing ledger.
- Changing contractor/rate data later cannot alter a met milestone contribution.

Scope guard: Do not add fixed-amount milestone splitting across contractors until a precise allocation rule is defined; the current hours-based engine is defensible.

M17 - Invoice Lifecycle Redesign: Billing Queue -> Vendor Draft -> Client Approval

Phase	Priority	Dependencies	Effort
Phase C - Commercial lifecycle	P1	M09,M15,M16,M01	XL

Objective: Correct the commercial ownership model: milestones create billing eligibility; Vendor creates/submits an invoice; Client/PM approves or rejects it.

Business value: The current Vendor approve/reject step is not a complete procure-to-pay workflow. This redesign makes invoice approval represent the client decision rather than vendor self-approval.

Current code evidence / starting point

- invoiceService currently auto-generates one invoice per milestone_billing
- vendorInvoiceService currently approves/rejects PENDING_REVIEW
- PM invoice API is read-only
- Original spec says Vendor generates invoice and Client views/validates it

Implementation work

Layer	Implementation
Billing queue	Stop auto-generating invoices from milestoneService. Milestone billings become an “eligible to invoice” queue.
Database	Normalize invoice header + invoice items. Each item references one milestone_billing_id with UNIQUE constraint so a billing contribution can never be invoiced twice.
Grouping	A draft invoice may group only compatible eligible billings (same vendor, client company, project, currency; define

Layer	Implementation
	milestone/billing-period rule explicitly).
Lifecycle	Use `DRAFT -> SUBMITTED -> APPROVED | REJECTED | CANCELLED`; keep payment state separate for M19. Rejected invoice can be revised/resubmitted under controlled version/history rules.
Vendor API	List un-invoiced billings, create draft, add/remove eligible items while DRAFT, submit invoice. Financial values come from immutable billings/rate snapshots, never client-provided totals.
PM API	List SUBMITTED invoices only for owned client projects; approve or reject with mandatory reason, race-safe row lock + conditional update.
Migration	Document mapping of current PENDING_REVIEW/APPROVED/AUTO_APPROVED/REJECTED demo rows; preserve old audit fields or migrate them into audit history. Do not pretend old vendor approval was client approval.
Frontend	Vendor billing queue/draft builder/submission; PM invoice review queue; clear lifecycle badges.
Notifications/Audit	Hook submission/review events into M14 and M04.
Tests	Double item claim, incompatible grouping, two client reviewers race, cross-tenant project access, reject/revise/resubmit, invoice totals equal item snapshots.

Acceptance criteria

- Milestone billing can exist without an invoice.
- Vendor explicitly creates/submits invoice.
- Only client-side PM can approve/reject submitted invoice.
- One milestone billing contribution can appear on at most one invoice item.

Scope guard: Do not allow users to type invoice totals independently of line items; totals must be derived from immutable billable records plus explicit tax/adjustment rules.

M18 - Invoice Numbers, Line Items, Tax & PDF Document

Phase	Priority	Dependencies	Effort
Phase C - Commercial lifecycle	P1	M17	L

Objective: Make an approved/submitted invoice a real business document rather than a JSON row.

Business value: Sequential identifiers, line items, tax and a printable PDF are what makes the billing module usable by finance teams and credible in a demo.

Current code evidence / starting point

- Current invoice rows have auto-increment id + amount/status only
- No invoice_number, invoice_items, tax fields or PDF generation exists
- Original spec explicitly shows INV-2026-001, subtotal, tax and total

Implementation work

Layer	Implementation
Numbering	Add immutable invoice_number generated from a vendor/year sequence under row lock, e.g. `INV-2026-000123`.
Line items	Each item shows contractor/role or skill, milestone/billing period, approved hours, bill rate and amount. Values come from milestone billing + assignment snapshot.
Tax	Add explicit tax rate/amount and optional adjustment lines; subtotal + tax + adjustments = total. Currency fixed per invoice.
Dates	Add invoice_date, submitted_at and due_date/payment_terms

Layer	Implementation
	fields.
PDF	Generate a server-side PDF from stored invoice data, store it via the same storage abstraction, and provide authorized download. Regenerate only while DRAFT; freeze the submitted/approved document version.
Frontend	Invoice detail/preview, PDF download, print-friendly view.
Tests	Sequential number race, exact decimal math, item totals, tax rounding, immutable document after submission, unauthorized download.

Acceptance criteria

- Every submitted invoice has a human-readable immutable number.
- PDF values exactly match database line items/totals.
- Invoice can be handed to a finance user without explaining internal database IDs.

Scope guard: Do not integrate an accounting suite yet; first make exportable, correct invoice documents.

M19 - Payment & Outstanding Tracking

Phase	Priority	Dependencies	Effort
Phase C - Commercial lifecycle	P1	M17,M18	M

Objective: Separate client approval from actual payment and expose outstanding/overdue money.

Business value: “Approved” is not “paid.” Vendors need to know what is due, partially paid and overdue.

Current code evidence / starting point

- Current invoice status ends at vendor APPROVED/REJECTED
- No payment table, paid amount, due date or outstanding calculation exists

Implementation work

Layer	Implementation
Database	Add `payments(invoice_id,amount,paid_at,reference,method,notes,recorded_by,created_at)`; index invoice/date.
Business rules	Payment can be recorded only against an APPROVED invoice; sum(payments) cannot exceed invoice total unless explicit credit/adjustment functionality is added later.
Derived status	Compute UNPAID/PARTIALLY_PAID/PAID from payment sum; compute OVERDUE from due_date + outstanding > 0. Keep this separate from invoice approval lifecycle status.
Backend	Vendor records payment received; PM can view payment status for own projects. Every payment is audited.
Frontend	Payment history, record-payment modal, outstanding amount, aging buckets and overdue indicators.
Notifications	Due-soon/overdue notifications can be added through M14 preferences.
Tests	Partial payments, exact payoff, overpayment rejection, payment rollback/audit and timezone-safe due-date calculation.

Acceptance criteria

- Dashboard can distinguish invoiced, approved, paid and outstanding.
- Partial payment history is preserved.
- Overdue is a deterministic derived fact, not a manually assigned status.

Scope guard: No payment gateway or banking integration; this module records settlement status only.

M20 - Business Dashboards, Financial Health & CSV Exports

Phase	Priority	Dependencies	Effort
Phase C - Commercial lifecycle	P2	M05,M10,M13,M15,M19	L

Objective: Make dashboards reflect the full workforce/commercial lifecycle with precise terminology and exportable operational data.

Business value: This is where the product becomes useful for day-to-day decisions: staffing gaps, approval bottlenecks, budget consumption, margin and cash outstanding.

Current code evidence / starting point

- Current Vendor/PM/Contractor dashboards already aggregate project progress, hours, invoices and revenue
- Current invoice “approved” terminology can be confused with actual payment
- No CSV export exists

Implementation work

Layer	Implementation
Vendor dashboard	Clients/projects, open requirements, candidate funnel/SLA, active/upcoming contractors, compliance expiry, approved hours, billable-uninvoiced, submitted/approved invoices, paid/outstanding/overdue, margin by client/project/skill.
PM dashboard	Project staffing funnel, pending candidate/timesheet/invoice reviews, SLA breaches, budget vs approved work vs invoiced vs paid, milestone progress.
Contractor dashboard	Active/upcoming assignments, allocation vs approved/submitted hours, rejected items requiring action, compliance expiries; if cost-rate/payable is modeled, show contractor payable facts instead of client invoice revenue.
Budget analytics	Budget utilization = approved/billable project cost against project budget; show planned/allocated/actual as distinct values.
Filters	Client/project/vendor/skill/status/date filters with server-side aggregation, not client-side full dataset downloads.
CSV	Export contractors/assignments, approved timesheets, invoice headers/items, payments and project financial summary. Use the same filters as the screen.
Performance	Add targeted aggregation indexes and query limits; inspect plans for the largest dashboard queries.
Tests	Metric definitions against known fixture data, terminology consistency, CSV totals matching API/dashboard values.

Acceptance criteria

- No dashboard uses “revenue/paid” interchangeably.
- Budget, billed, approved, paid and outstanding are distinct.
- Finance/operations can export filtered CSV that reconciles to on-screen totals.

Scope guard: Do not add opaque financial health scores or AI recommendations; transparent deterministic metrics are stronger here.

M21 - Contractor Offboarding, History & Project Close Safeguards

Phase	Priority	Dependencies	Effort
Phase D - Portfolio-grade operations	P2	M11,M19,M04	M

Objective: Close assignments/projects cleanly without losing historical workforce or billing records.

Business value: Real workforce management includes release reasons, unresolved timesheets and billing checks; ending work should not simply flip a flag.

Current code evidence / starting point

- Current project completion releases all active assignments
- No individual offboarding checklist/history view exists
- Historical assignment/timesheet/billing rows are intentionally preserved already

Implementation work

Layer	Implementation
Assignment	Extend release with planned/actual last working date, release reason and actor.
Checks	Before release/project completion, surface submitted/rejected timesheets, unbilled milestone contributions, draft/submitted invoices and unpaid approved invoices. Decide which are blockers vs warnings explicitly.
History	Contractor history page: past assignments, client/project, dates, skills/role, approved hours, rate snapshot and billing/payment facts authorized to Vendor.
Project close	Completion checklist confirms staffing released, approvals handled and billing status understood; project status change remains explicit, never auto-triggered by hours.
Audit	All close/release actions recorded.
Tests	Release with pending work, project completion with unresolved records, historical visibility after contractor status INACTIVE.

Acceptance criteria

- Individual release and project completion explain unresolved work instead of silently hiding it.
- Historical assignment and financial records remain queryable after offboarding.

Scope guard: Do not turn contractor history into a subjective performance score.

M22 - Production Deployment, Backup, Security & Demo Environment

Phase	Priority	Dependencies	Effort
Phase D - Portfolio-grade operations	P2	M01,M02,M03,M20	L

Objective: Make the expanded system reproducible and demonstrable outside the development machine.

Business value: A portfolio project becomes much stronger when another engineer can run or open it, test all three roles, and see realistic seeded workflows.

Current code evidence / starting point

- Current repo is local Node/Vite/MySQL oriented
- No complete production deployment definition is checked in
- Current README is stale relative to the implemented modules

Implementation work

Layer	Implementation
Containers	Add Dockerfiles and docker-compose for backend/frontend/MySQL in local demo; migration command must run explicitly and idempotently.
Environment	Separate dev/test/prod config, secure secret handling, strict production CORS, cookie/security settings and storage configuration.
Database ops	Document backup/restore, migration rollback strategy where possible, and test a restore into a fresh environment.
Demo data	Seed a deterministic scenario with Vendor, PM/client, contractors, requirements, candidate submissions,

Layer	Implementation
	approved/rejected timesheets, milestones, invoice and payment states.
E2E	Add Playwright smoke journeys for Vendor -> candidate submit, PM accept, contractor timesheet, PM review, vendor invoice submit, PM approve, payment record.
Performance	Basic load checks on list/search and concurrent approval/assignment hotspots.
Security review	Dependency audit, headers, file upload limits, authorization matrix and secret scan.
Deployment	Host one demo environment or provide one-command local demo; keep provider-specific choices outside core domain code.

Acceptance criteria

- Clean checkout can start the complete system with documented commands.
- Backup restore is tested.
- A reviewer can exercise the main end-to-end lifecycle with seeded accounts/data.

Scope guard: Do not add Kubernetes/microservices for a single Node API + MySQL + React application.

M23 - Final Architecture Audit, Release & Internship Story

Phase	Priority	Dependencies	Effort
Phase D - Portfolio-grade operations	P2	M00-M22	M

Objective: Validate that the expanded platform is coherent, tested and explainable, then cut a portfolio release.

Business value: The value of the project during internship comes from being able to defend each business rule and engineering choice, not just showing many screens.

Current code evidence / starting point

- Project began as a two-day Workday hackathon MVP and already contains documented revisions/fixes
- The user wants to present post-hackathon independent expansion honestly

Implementation work

Layer	Implementation
Audit	Re-review every migration/service/repository/route/frontend workflow against the final architecture and original spec; list deliberate deviations.
Security	Authorization matrix test across Vendor/Contractor/PM, including all new client/project/candidate/document/invoice/payment resources.
Data integrity	Reconcile approved hours -> milestone billings -> invoice items -> invoice totals -> payments; add one automated reconciliation test.
Docs	Final architecture diagram, ERD, business workflow diagram, API docs, deployment instructions and decision log.
Release	Tag a stable release (e.g. `v2.0-portfolio` or semantic version chosen at that time) and write release notes separating hackathon MVP from independent extensions.
Demo	Prepare a 5-8 minute role-switch demo and 2-3 deeper engineering stories: concurrency/locking, immutable financial snapshots/idempotency, and a business-workflow redesign such as candidate acceptance or invoice lifecycle.
Resume/interview	Describe truthfully: initial MVP built in Workday hackathon; independently audited, hardened and extended after the event.

Acceptance criteria

- All critical workflows have automated + manual E2E coverage.
- Documentation and demo match actual code.
- You can explain why each major change exists and what invariant protects it.

Scope guard: Do not add last-minute AI or unrelated modules just to increase feature count before the internship.

8. Critical Dependency Logic

- Build tests/CI before changing business workflows. M01 protects every later refactor.
- Build audit foundations before the new workflow modules so new state transitions are auditable from day one.
- Fix contractor identity/skills/compliance and vendor-client project scoping before candidate submission; otherwise the candidate module would be built on weak eligibility and tenancy assumptions.
- Fix project lifecycle and date-based assignment before candidate acceptance because acceptance must be able to revalidate requirement capacity, project dates, compliance and overlap atomically.
- Build rate cards before the final invoice document model so invoice items can use stable assignment/billing snapshots without another financial migration.
- Keep milestone billings as the immutable billing-eligibility ledger; invoice items should consume those billings, not recompute approved hours/rates.
- Payment tracking must come after client invoice approval because payment is a settlement fact, not an approval state.
- Dashboard financial analytics comes after budget/rates/invoices/payments so the terminology and formulas can reconcile to authoritative records.
- Production deployment and portfolio release come last, after core workflow/state models stabilize.

9. Explicit Do-Not-Build List (for this roadmap)

- AI resume/candidate scoring or automated rejection
- LLM chatbot on every screen
- Predictive attrition/performance scoring
- Facial attendance
- Payroll engine or employee HRMS
- Full ATS/video interview system
- Generic CRM/sales pipeline
- Payment gateway/bank integration
- Blockchain invoices
- Microservices/Kubernetes rewrite
- ERP/accounting replacement
- Fractional overlapping contractor assignments before the no-overlap model is proven
- Fixed-amount milestone splitting before a precise multi-contractor allocation rule exists

10. How This Becomes a Strong Internship/Portfolio Story

The truthful story should stay simple: the initial MVP was built during the Workday two-day hackathon; after the hackathon, the codebase was independently audited, hardened and expanded into a more complete VMS. The value is not the feature count. The value is that each extension solves a real workflow problem and preserves data/concurrency correctness.

- Business story: direct assignment became candidate submission + client acceptance; one-active-assignment became date-aware availability; vendor self-review became vendor invoice submission + client approval + payment tracking.
- Engineering story: the project uses transactions, row locks, unique constraints and immutable snapshots to keep staffing and billing correct under concurrency.
- Quality story: manual scripts were converted into automated regression/integration tests and CI before major refactors.
- Product story: compliance, audit trail, SLA, notifications, client scoping, invoice documents and exports make the platform closer to an operational vendor workflow rather than a demo CRUD app.

11. Source Basis Used for This Plan

- Uploaded code ZIP: `9b4728b9-7b96-4b11-b543-f709ac2d1d44.zip` (`workday-master`).
- Original embedded specification: `workday-master/Vendor Management.pdf`.
- Backend migrations 001-016; routes, services, repositories, validators and current manual regression scripts.
- Frontend React route tree, role pages, dashboards, timesheet grouping and API service modules.
- The consolidated VMS code-review/roadmap text supplied in the conversation and prior Workday hackathon module decisions.